

CSC1029 OO Programming

Lecture 18 – Inheritance and Method Overriding



The Story So Far . . . (1)

We started a few weeks back by looking at how we can define **object classes** and some simple **associations** between them.

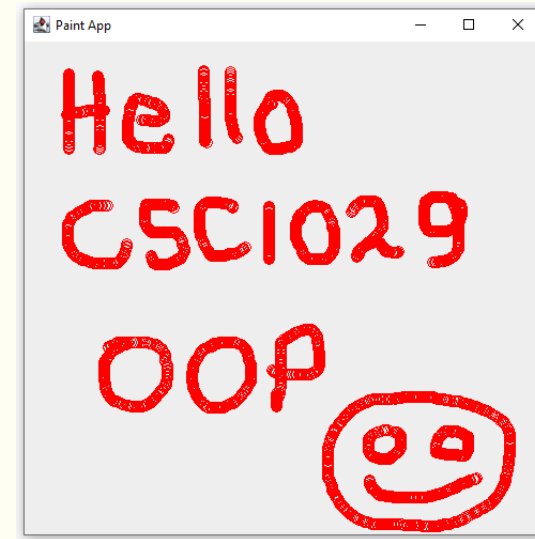
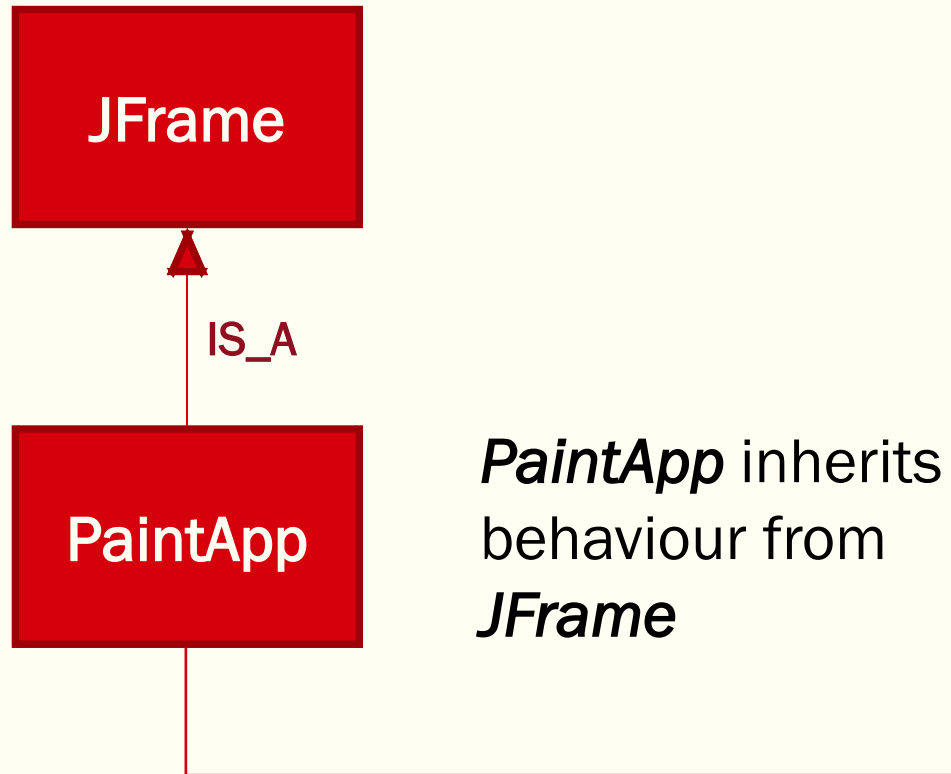
We use these classes to **instantiate** objects used with an application.

But so far ... we have had to do all the coding.

Inheritance enables the developer to reuse code – enabling the rapid development of more complex systems – without the need to start from scratch.

Inheritance is a fundamental component of OOP – defined by the ‘**IS_A**’ relationship.

The Story So Far . . . (2)



PaintApp objects belong to the more general group (class) of *JFrame* objects

What's to come today?

- BankAccount Example
- Method Overriding
- Runtime Polymorphism Introduction

Inheritance in Java (1)

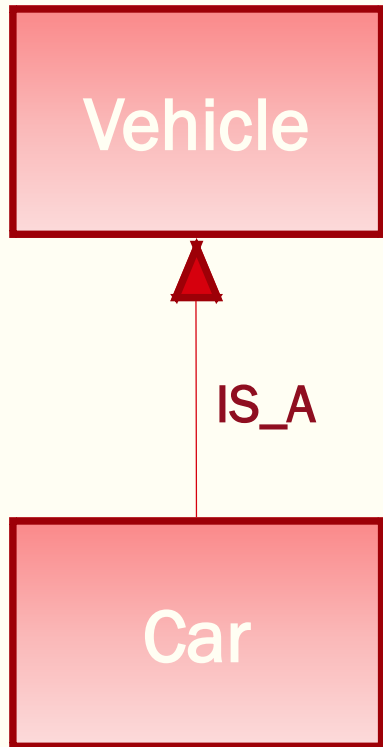
- BankAccount Example
- Method Overriding
- Runtime Polymorphism Introduction

Inheritance in Java (1)

```
class SuperclassName{  
    //states and behaviours  
}
```

```
class SubclassName extends SuperclassName{  
    //states and behaviours  
}
```

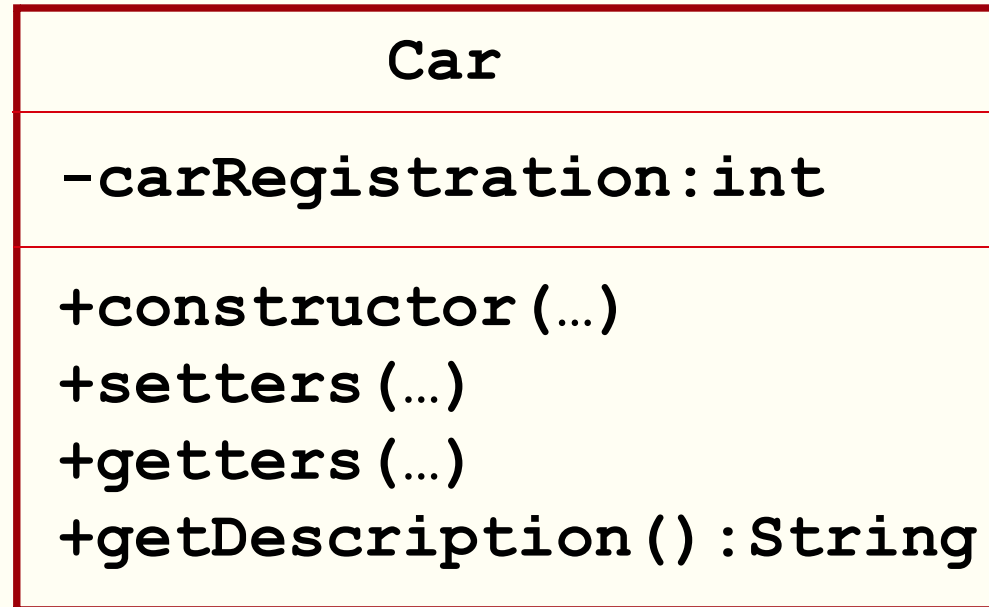
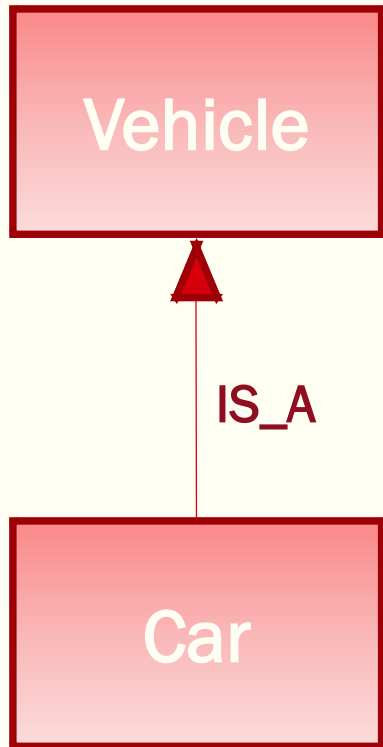
Inheritance in Java (2)



Vehicle
-numberOfTyres:int -engineSize:double -colour:String
+constructor (...) +setters (...) +getters (...) +printDetails ():String

Vehicle.java

Inheritance in Java (3)



Car.java

CarTester.java

Inheritance in Java (4)

Inheritance is a **one way** relationship ...

- A child inherits from a parent
- NOT the other way round

If you instantiate a variable of type Vehicle, it **does not** have access to the Car class states and behaviours

If you instantiate a variable of type Car, it does have access to the Vehicle class states and behaviours

Subclasses are more specific than the superclass they extend

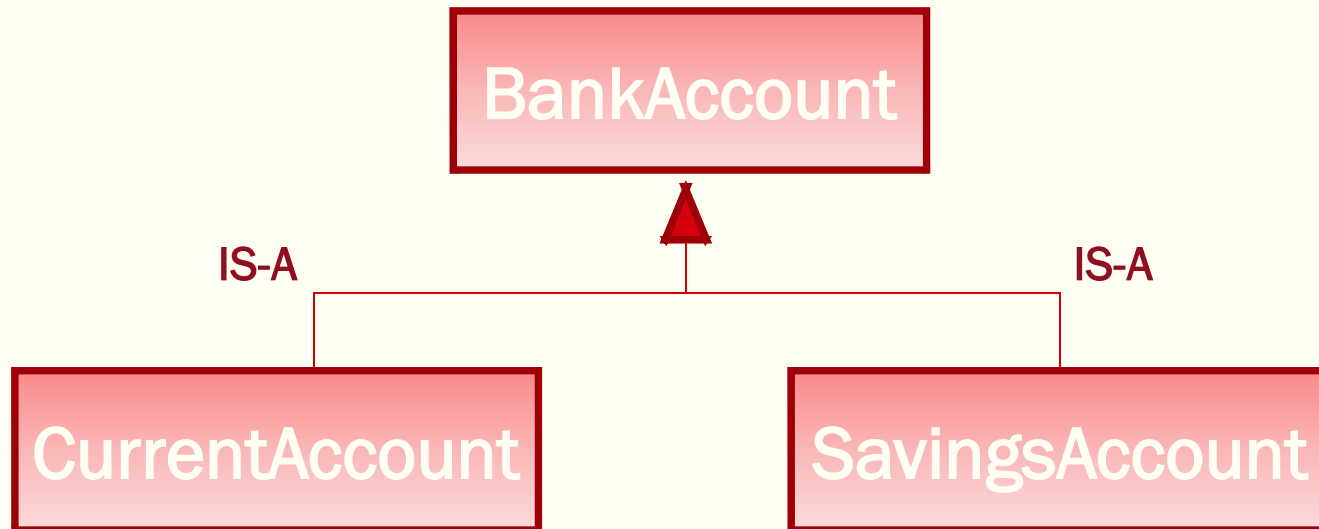
Inheritance in Java (5)

There is an `instanceOf` operator in Java which tells us if an instance of an object is a member of a particular class

```
Car myMini = new Car(...);  
Vehicle any = new Vehicle(...);  
Scanner input = new Scanner(...);  
  
boolean answer1 = myMini instanceof Vehicle;  
boolean answer2 = any instanceof Car;  
boolean answer3 = input instanceof Vehicle;
```

See: *CarInstanceOfTester.java*

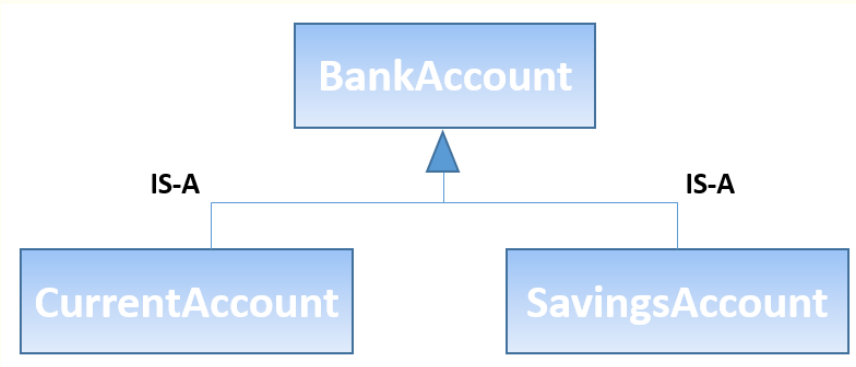
Inheritance in Java – BankAccount (1)



Assume BankAccount is defined within package Bank and subclasses are developed elsewhere.

BankAccount
- name : String - acctNum : int - balance : double - <u>nextNum : int = 1</u>
+BankAccount(String) +deposit(double):boolean +withdraw(double):boolean +statement():String - <u>nextAcctNum():int</u>

Inheritance in Java – BankAccount (2)



BankAccount
- name : String - acctNum : int - balance : double - <u>nextNum : int = 1</u>
+BankAccount(String) +deposit(double):boolean +withdraw(double):boolean +statement():String - <u>nextAcctNum():int</u>

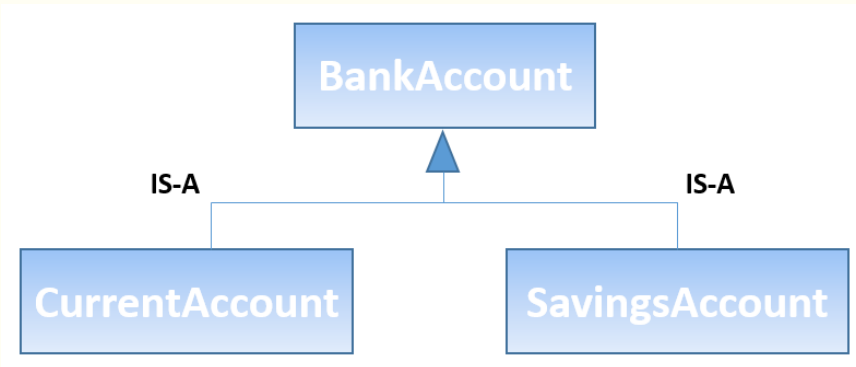
For any proposed **subclass** of BankAccount – what is inherited/visible?

Private methods/data of the **superclass** are **not visible** within a **subclass**.

Public methods/data of the **superclass** are **visible** within a **subclass**:

constructor
deposit()
withdraw()
statement()

Inheritance in Java – BankAccount (3)

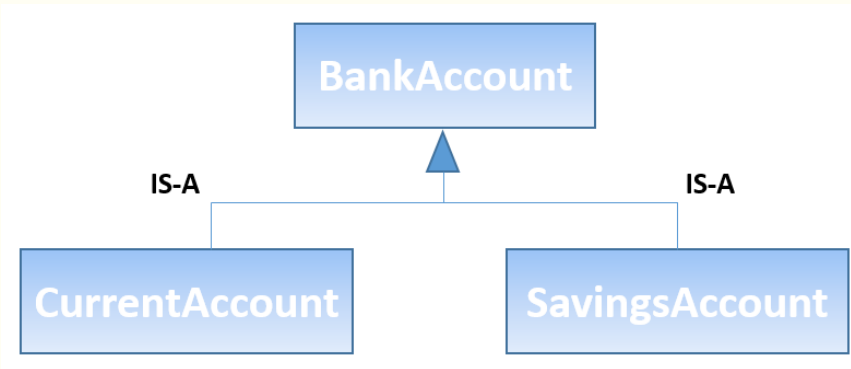


What specific attributes/behaviour could be added to subclass *SavingsAccount*?

BankAccount
- name : String - acctNum : int - balance : double - <u>nextNum : int = 1</u>
+BankAccount(String) +deposit(double):boolean +withdraw(double):boolean +statement():String -<u>nextAcctNum():int</u>

SavingsAccount
- interestRate : double
+ SavingsAccount(String) + getInterestRate(): double + setInterestRate(double):void + payInterest():void

Inheritance in Java – BankAccount (4)



SavingsAccount
- interestRate : double
+ SavingsAccount(String) + getInterestRate(): double + setInterestRate(double):void + payInterest():void

```
public class SavingsAccount extends BankAccount{

    private double interestRate;    // as a %

    public SavingsAccount(String name) {
        super(name);
        this.interestRate = 0.0;
    }

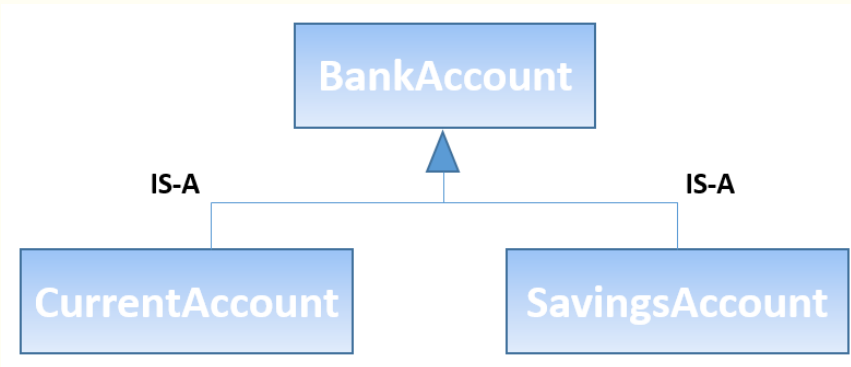
    ...
}
```

A *SavingsAccount* object IS_A *BankAccount* ...

Constructor of *SavingsAccount* must call the constructor of *BankAccount* through the *super* reference.

Must be done in the first line of the *SavingsAccount* constructor.

Inheritance in Java – BankAccount (5)



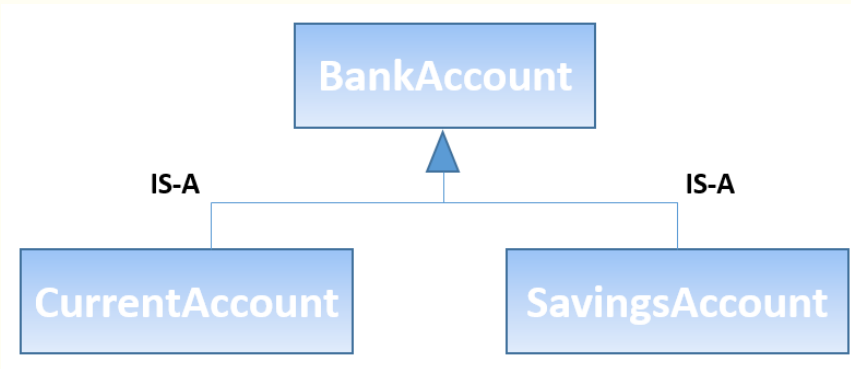
SavingsAccount
- interestRate : double
+ SavingsAccount(String) + getInterestRate(): double + setInterestRate(double):void + payInterest():void

```
public double getInterestRate() {  
    return interestRate;  
}
```

```
public void setInterestRate(double rate) {  
    if ( rate > 0.0 ) { // pre-condition  
        this.interestRate = rate;  
    }  
}
```

```
public void payInterest() {  
    if ( interestRate > 0.0 ) {  
        double amt = (getBalance()*interestRate)/100.0;  
        deposit(amt); // inherited method  
    }  
}
```

Inheritance in Java – BankAccount (6)



SavingsAccount

- interestRate : double

+ SavingsAccount(String)
+ getInterestRate(): double
+ setInterestRate(double):void
+ payInterest():void

// a savings account

```
SavingsAccount sal = new SavingsAccount("Bart");
sal.setInterestRate(4.0); // 4% interest
sal.deposit(20.0); // deposit some money
```

```
System.out.println(sal.statement()); // inherited
sal.payInterest(); // payment of interest
```

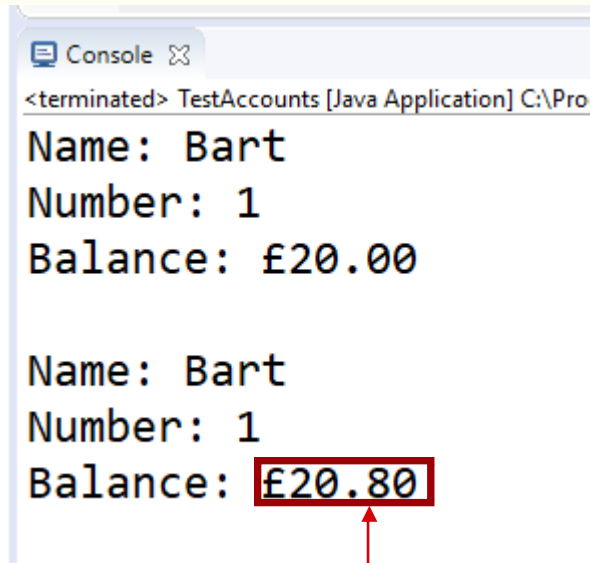
// another statement ...

```
System.out.println(sal.statement());
```

```
Console
<terminated> TestAccounts [Java Application] C:\Pro
Name: Bart
Number: 1
Balance: £20.00

Name: Bart
Number: 1
Balance: £20.80
```


Formatted Output



```
<terminated> TestAccounts [Java Application] C:\Pro
Name: Bart
Number: 1
Balance: £20.00

Name: Bart
Number: 1
Balance: £20.80
```

How do we achieve
formatted output?

There are a number of ways to do this Java, including `String.format(...)` and `System.out.printf(...)`

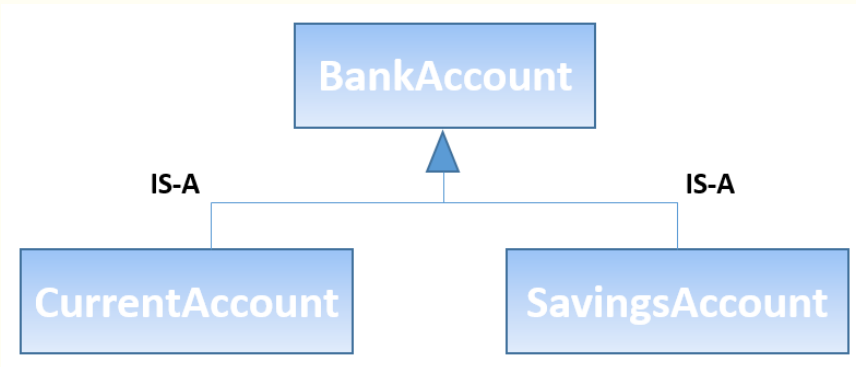
```
// taken from class BankAccount (superclass)
public String statement() {
    String result = "Name: " + getName() + "\n";
    result += "Number: " + getAcctNum() + "\n";
    result += "Balance: " +
        String.format("£%.2f", getBalance()) + "\n";
    return result;
}
```

Produces a String Format Required Value to be Formatted

%.2f

Placeholder for a value (float/double) – to be displayed with 2 digits after the decimal point

Inheritance in Java – BankAccount (1)



BankAccount
- name : String - acctNum : int - balance : double - <u>nextNum : int = 1</u>
+BankAccount(String) +deposit(double):boolean +withdraw(double):boolean +statement():String - <u>nextAcctNum():int</u>

What specific attributes/behaviour could be added to subclass *CurrentAccount*?

CurrentAccount
- overdraftLimit : double
+ CurrentAccount(String) + getOverdraftLimit(): double + setOverdraftLimit(double):void

Potential problem – *withdraw()* from BankAccount does not allow a balance below £0.

Inheritance in Java – BankAccount (2)

Potential problem – *withdraw()* from *BankAccount* does not allow a balance below £0.

<terminated> testAccounts [Java Applica

Name: Homer
Number: 1
Balance: £25.00

Name: Homer
Number: 1
Balance: £25.00

So trying to withdraw £40 had no effect.

```
// a current account
CurrentAccount ca1 = new CurrentAccount("Homer");
// set overdraft limit
ca1.setOverdraftLimit(50.0);           // can go £50
                                       // into the red

// deposit some money
ca1.deposit(25.0);

// call inherited statement() method
System.out.println(ca1.statement());

// withdraw £40 - should go £15 into the red
ca1.withdraw(40.0); // could also test result

// get another statement
System.out.println(ca1.statement());
```

Inheritance in Java – BankAccount (3)

Potential problem – *withdraw()* from **BankAccount** does not allow a balance below £0.

```
<terminated> testAccounts [Java Applica  
Name: Homer  
Number: 1  
Balance: £25.00  
  
Name: Homer  
Number: 1  
Balance: £25.00
```

What can we do about this?

Perhaps we could change the behaviour of *withdraw()* in **BankAccount** to permit overdrafts?

This would be a very bad idea – why?

Well, for a start, it would mean that any **SavingsAccount** object would now be able to go into the red!

So – avoid changing the behaviour/public interface for potential **superclass** users

Any other suggestions?

Inheritance in Java – Method Overriding (1)

Fortunately, OOP enables a subclass to *override* an existing method definition.

```
public class BankAccount {  
    ...  
    public boolean withdraw(double amt) {  
        if (amt>0.0 && (balance-amt>=0.0)) {  
            balance -= amt;  
            totalAmt -= amt;  
            return true;  
        }  
        return false;  
    }  
}
```

But how does this help?

Inheritance in Java – Method Overriding (2)

To override *withdraw()* from *BankAccount* – we need a method with the same signature in *CurrentAccount*.

```
public class CurrentAccount extends BankAccount {  
    ...  
    @Override  
    public boolean withdraw(double amt) {  
        // pseudocode  
        // if (amt > 0 and (balance-amt) within overdraft limit)  
        //     take amt away from balance  
        //     return true  
        // return false  
    }  
}
```

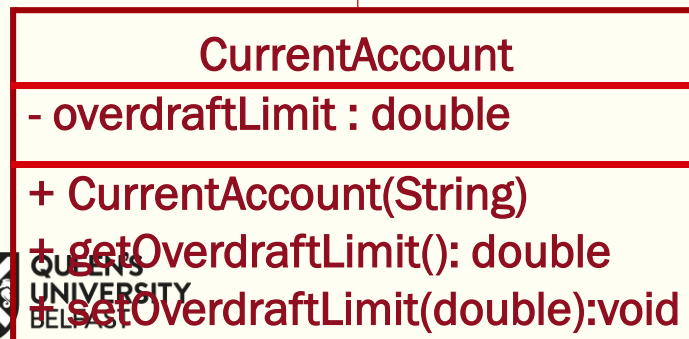
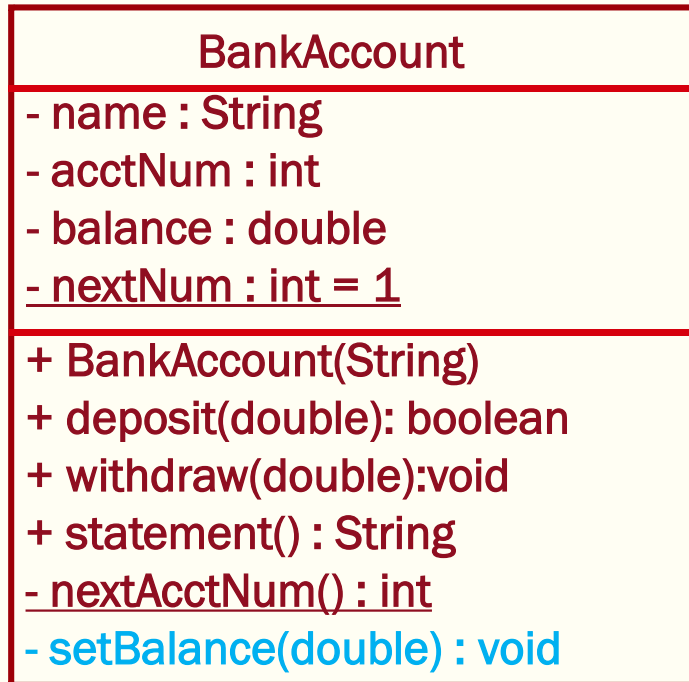
Inheritance in Java – Method Overriding (3)

To override *withdraw()* from *BankAccount* – we need a method with the same signature in *CurrentAccount*.

```
public class CurrentAccount extends BankAccount {  
    ...  
    @Override  
    public boolean withdraw(double amt) {  
        if (amt > 0.0 && (balance-amt) >= -overdraftLimit) {  
            // take amt away from balance  
            return true  
        }  
        return false  
    }  
}
```

- To do this – we need to be able to set the balance.
- But this facility is not available.
- What do we do now?

Inheritance in Java – Method Overriding (4)



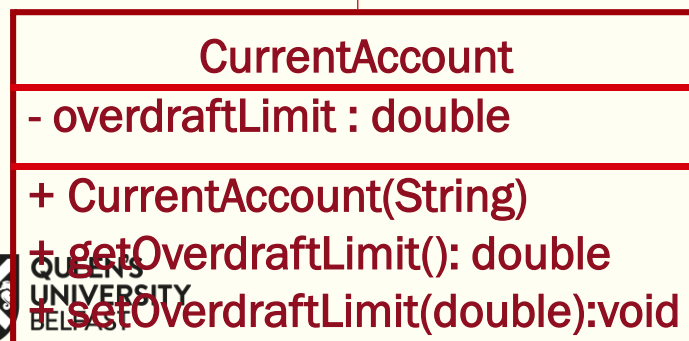
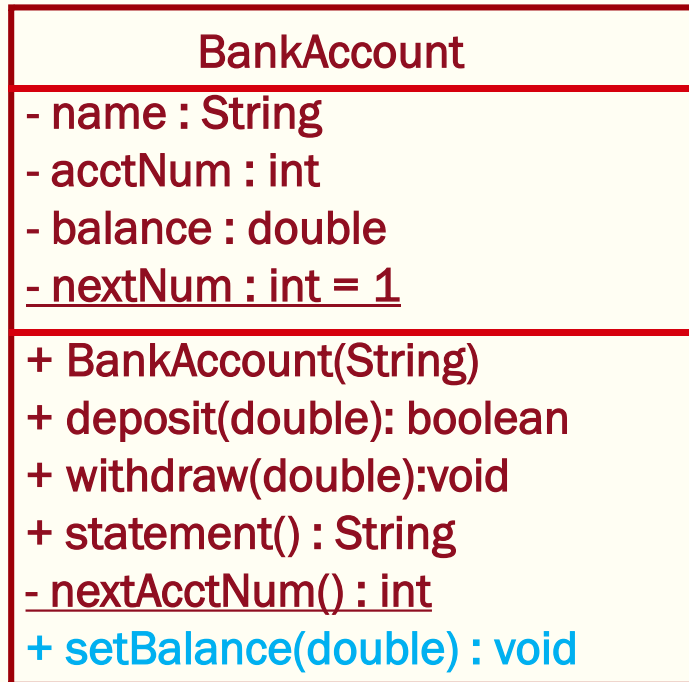
How do we provide a 'safe' mechanism to set the balance for a bank account?

```
BankAccount ba1 = new BankAccount("Fred");
```

```
ba1.setBalance(500.0); //illegal as method private  
// This is what we want - no change to public  
// interface of BankAccount.
```

```
CurrentAccount ca1 = new CurrentAccount("Paul");  
ca1.deposit(200.0);  
ca1.setOverdraftLimit(100.0);  
ca1.withdraw(250.0); // will not work  
// withdraw will not have access to setBalance()
```


Inheritance in Java – Method Overriding (5)



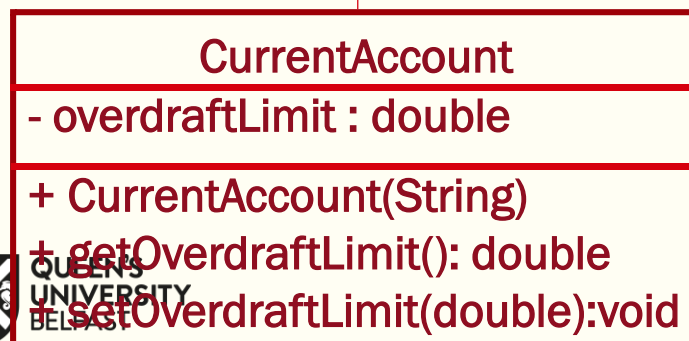
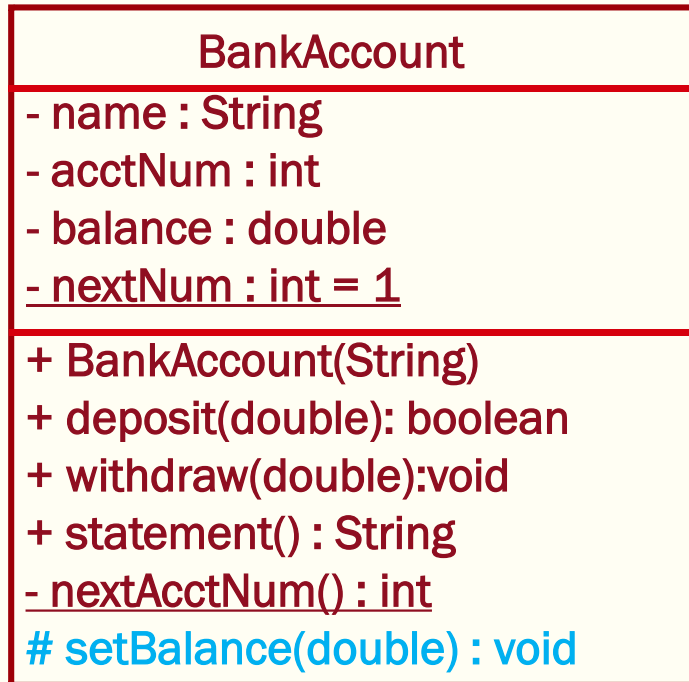
How do we provide a 'safe' mechanism to set the balance for a bank account?

```
BankAccount ba1 = new BankAccount("Fred");
```

```
ba1.setBalance(500.0); //works as method now  
// public - but we should NOT allow this
```

```
CurrentAccount ca1 = new CurrentAccount("Paul");  
ca1.deposit(200.0);  
ca1.setOverdraftLimit(100.0);  
ca1.withdraw(250.0); // will now work
```

Inheritance in Java – Method Overriding (6)



How do we provide a 'safe' mechanism to set the balance for a bank account?

```
BankAccount ba1 = new BankAccount("Fred");
```

```
ba1.setBalance(500.0); //illegal as method not  
// part of public interface – what we require.
```

The access modifier for setBalance() is *protected*

```
CurrentAccount ca1 = new CurrentAccount("Paul");  
ca1.deposit(200.0);  
ca1.setOverdraftLimit(100.0);  
ca1.withdraw(250.0); // will now work  
// as access to setBalance() is passed on to  
// the subclass
```

Inheritance in Java – Method Overriding (7)

We arrive at a solution which **overrides** `withdraw()` – access to ***protected*** data/methods is passed on to a subclass – and viewed as ***private*** at this level – i.e. can be accessed internally.

```
public class CurrentAccount extends BankAccount {  
    ...  
    @Override  
    public boolean withdraw(double amt) {  
        if (amt>0.0 && (getBalance()-amt) >= -overdraftLimit) {  
            setBalance( getBalance() - amt);  
            return true;  
        }  
        return false;  
    }  
}
```

This is another example of ***polymorphic*** behaviour – ***runtime polymorphism***.

Inheritance in Java – Method Overriding (8)

We arrive at a solution which *overrides* `withdraw()` – access to *protected* data/methods is passed on to a subclass – and viewed as *private* at this level – i.e. can be accessed internally.

```
public class BankAccount {  
    ...  
    protected void setBalance(double amt) {  
        balance = amt;  
    }  
}
```

Use the *# symbol* in *UML* to indicate a *protected* access modifier.

Inheritance in Java – Method Overriding (9)

Overridden method *withdraw()* in *CurrentAccount* will now permit an overdraft.

```
terminated testAccounts.java Application
Name: Homer
Number: 1
Balance: £25.00
|
Name: Homer
Number: 1
Balance: £-15.00
```

```
// a current account
CurrentAccount ca1 = new CurrentAccount("Homer");
// set overdraft limit
ca1.setOverdraftLimit(50.0);           // can go £50
                                       // into the red

// deposit some money
ca1.deposit(25.0);

// call inherited statement() method
System.out.println(ca1.statement());

// withdraw £40 - should go £15 into the red
ca1.withdraw(40.0); // could also test result

// get another statement
System.out.println(ca1.statement());
```

Inheritance in Java – Method Overriding (10)

Using method **overriding** – we can also refine the use of the **statement()** method – making specific changes for each **subclass**.

For example, we can **override statement()** in **SavingsAccount** ...

```
@Override
public String statement() {
    String res = "Savings Account Statement\n";
    res += "+++++\n";
    res += super.statement();
    res += "Interest Rate: " + interestRate;
    return res;
}
```

```
<terminated> testAccounts [Java Application] C:\Program Files\Java\
Savings Account Statement
+++++
Name: Bart
Number: 1
Balance: £20.00
Interest Rate: 4.0

After interest payment....

Savings Account Statement
+++++
Name: Bart
Number: 1
Balance: £20.80
Interest Rate: 4.0
```

Inheritance in Java – Method Overriding (11)

Using method **overriding** – we can also refine the use of the **statement()** method – making specific changes for each **subclass**.

For example, we can **override statement()** in **CurrentAccount** ...

```
@Override
public String statement() {
    String res = "Current Account Statement\n";
    res += "+++++\n";
    res += super.statement();
    res += "Overdraft Limit: " + overdraftLimit;
    return res;
}
```

<terminated> TestAccounts [Java Application] C:\Program Files\J

```
Current Account Statement
+++++
Name: Homer
Number: 1
Balance: £25.00
Overdraft Limit: 50.0
```

After withdrawing £40....

```
Current Account Statement
+++++
Name: Homer
Number: 1
Balance: £-15.00
Overdraft Limit: 50.0
```



What's Next?

Runtime Polymorphism
Abstract Classes
Interfaces